MASTER DOCUMENTATION

Book My Stars Platform Guide

Complete Architecture, Spring Boot Classes, Annotations & API Reference

📑 Table of Contents

1. Frontend Routing & State Architecture
2. Backend Classes & Annotations Deep-Dive
3. Complete API Endpoints Catalog
4. End-to-End Request Journey & Navigation

1. Routing & State Management

How navigation and data synchronization work across the React application.

Routing Architecture (React Router DOM v7)

react-router-dom

The application uses `<BrowserRouter>` in `main.jsx` leveraging HTML5 History API for seamless client-side single page routing.

Public Routes: `/` (Landing Page), `/become-client`, `/become-partner`, `/podcast`, `/public-profile/:userId`

Auth Routes: `/signin`, `/join` (Multi-step wizard)

Protected Routes: `<ProtectedRoute>` guards `/profile`, `/my-media`, `/dashboard`, `/auditions`, `/bookings`, `/calendar`, `/messages`, `/contacts`. Redirects unauthorized users to `/`.

Smart History: On logout, `window.history.replaceState(null, "", "/")` ensures browser Back button never re-enters private pages.

4-Tier State Management

Zero Redux Bloat

Clean, high-performance hybrid state architecture:

1. Local Component State (`useState`): Controls UI tabs (Overview, Media, About), modals, cropper, and form inputs.

2. Persistent Storage (`localStorage`): `bms_token` (JWT header), `bms_user` (cached identity), `bms_profile_photo` (instant avatar load without flash).

3. Reactive Event Bus (`window.dispatchEvent`): Non-parent-child updates via custom events: `profile_photo_updated` , `portfolio_updated` , `videos_updated` , `work_experience_updated` .

4. Server State & API Service: Centralized in `src/services/api.js` with optimistic UI updates.

2. Backend Classes & Annotations

Top-to-bottom breakdown of what each Java class does and why annotations are used.

BookmystarsApplication.java

Bootstrap

Application entry point that boots Spring context and embedded Tomcat on port 8080.

`@SpringBootApplication`

`main(String[] args):` Calls `SpringApplication.run(...)` to scan components and launch the server.

SecurityConfig.java

Security

Defines endpoint access rules, stateless JWT filter chain, password hashing, and CORS.

`@Configuration` `@EnableWebSecurity` `@Bean`

`passwordEncoder():` Returns `BCryptPasswordEncoder` for secure one-way salted password hashing.

`securityFilterChain(...):` Disables CSRF (stateless API), whitelists `/api/auth/**` & `/uploads/**` , protects everything else, and injects `JwtAuthenticationFilter` .

`corsConfigurationSource():` Allows Vite frontend (port 5173) to communicate without browser CORS errors.

JwtService.java & JwtAuthenticationFilter.java

JWT Auth

Generates, validates, and intercepts JSON Web Tokens for stateless authentication.

`@Service` `@Component` `@RequiredArgsConstructor`

`generateToken(email):` Signs HMAC-SHA256 token containing email and 24h expiration timestamp.

`doFilterInternal(...):` Intercepts every request, extracts `Authorization: Bearer` token, verifies signature, and authenticates the user in `SecurityContextHolder` .

ProfileController.java & ProfileService.java

Profile & Sharing

Central hub for talent bio, rates, headshot uploads, public URLs, and multi-channel sharing.

`@RestController` `@RequestMapping("/api/profile")` `@Service`

`getMyProfile()`: Returns full talent stats (credits, features, rates, about).

`uploadProfilePhoto()`: Saves image to `uploads/profile/`, updates `Profile`, and copies headshot into `Portfolio`.

`getShareProfile()`: Returns pre-filled URLs for WhatsApp, Gmail, Outlook Web, and ASCII-safe `mailto:` link (<400 chars).

`getPublicProfile(userId)`: Public endpoint aggregating bio, photos, reels, and credits for casting directors.

WorkExperienceController.java & WorkExperienceService.java

Experience

Handles acting credits and brand endorsements with lenient date handling.

`@RestController` `@Service` `@Valid`

`parseDate(input)`: Lenient date parser accepting 4-digit years (`"2024"`), ranges (`"2022 - 2024"`), ISO dates, or empty strings without throwing crashes.

`addExperience(...), getMyExperiences(...)`: CRUD operations on `work_experiences` table.

Media Controllers (Portfolio, Video, Audio, Document)

Media Assets

Unified modules for storing and serving photos, 16:9 reels, voice demos, and resume documents.

`@RestController` `@RequestParam MultipartFile`

Validates file formats (JPG, PNG, MP4, MP3, PDF, DOCX), generates UUID filenames, saves to physical disk under `uploads/`, and creates database records.

3. Complete API Catalog

Comprehensive reference of all endpoints, HTTP methods, and required payloads.

METHOD	ENDPOINT	ACCESS	DESCRIPTION & PAYLOAD
POST	`/api/auth/register`	Public	RegistrationRequest (fullName, email, password, DOB, gender, profileTypes)
POST	`/api/auth/login`	Public	LoginRequest (email, password) -> Returns JWT token + User info
GET	`/api/profile/me`	Auth	Fetches authenticated talent profile, bio, rates, and stats
PUT	`/api/profile/me`	Auth	ProfileUpdateRequest (about, languages, dayRate, experience, location)
POST	`/api/profile/photo`	Auth	Multipart `file` : Uploads avatar and syncs to portfolio
GET	`/api/profile/share`	Auth	Returns WhatsApp, Gmail, Outlook, and Mailto sharing URLs
GET	`/api/profile/public/{id}`	Public	Complete public portfolio and credits view for visitors
GET	`/api/portfolio/my`	Auth	Fetches all uploaded talent photos
POST	`/api/portfolio/upload`	Auth	Multipart `file` , `title` , `category` : Uploads photo
GET	`/api/profile/videos`	Auth	Fetches user showreels and video audition clips
POST	`/api/profile/videos/upload`	Auth	Multipart `file` : Uploads MP4/MOV showreel file
POST	`/api/profile/videos/link`	Auth	VideoRequest: Saves external YouTube/Vimeo video link
GET	`/api/profile/audio`	Auth	Fetches audio voiceover demos and singing clips
POST	`/api/profile/audio/upload`	Auth	Multipart `file` : Uploads MP3/WAV audio track
GET	`/api/profile/documents`	Auth	Fetches resume, CV, and contract documents
POST	`/api/profile/documents/upload`	Auth	Multipart `file` : Uploads PDF/DOCX resume file
GET	`/api/profile/work-experience`	Auth	Lists all acting and modeling credits
POST	`/api/profile/work-experience`	Auth	WorkExperienceRequest (company, title, category, startDate, endDate)
GET	`/api/profile/measurements`	Auth	Fetches physical casting stats (height, chest, waist, shoes)

METHOD	ENDPOINT	ACCESS	DESCRIPTION & PAYLOAD
POST	`/api/profile/measurements`	Auth	MeasurementRequest: Saves/updates physical casting stats
GET	`/api/profile/availability`	Auth	Fetches calendar schedule (AVAILABLE, BOOKED, TENTATIVE)
GET	`/api/profile/rates`	Auth	Lists pricing packages (Full day, commercial, hourly)
GET	`/api/profile/reviews/{id}`	Public	Returns client ratings and computed average score

4. End-to-End Request Journey

Trace of what happens when a button is clicked in React until PostgreSQL responds.

1

Trigger in React UI

User clicks a button (e.g. "Share Profile" or "Save Bio"). React invokes an asynchronous function in `src/services/api.js`.

2

Token Injection via api.js

`getAuthHeaders()` reads `localStorage.getItem("bms_token")` and attaches `Authorization: Bearer <JWT>` to the fetch request.

3

Security Filter Interception

`JwtAuthenticationFilter` intercepts the request before any controller. It verifies the HMAC-SHA256 signature, validates the timestamp, and stores the authenticated user into `SecurityContextHolder`.

4

Controller Routing

Spring's `DispatcherServlet` routes the request to the matching `@RestController` method (e.g., `ProfileController.getShareProfile`), injecting `Authentication auth`.

5

Service Business Logic

The controller delegates to `@Service` (e.g., `ProfileService`). It validates data, formats strings, and prepares URLs.

6

Repository & Database Execution

`@Repository` executes Spring Data JPA queries. Hibernate generates SQL, queries PostgreSQL (port 5432), and hydrates the returned rows into Java entity objects.

7

JSON Serialization & UI Re-Render

Jackson converts the Java DTO into a JSON response with HTTP 200 OK. The Promise resolves in React, `useState` updates, and the UI re-renders with fresh data.

Book My Stars © 2026 • High-Performance Casting & Talent Management Platform